PYTHON FOR BEGINNERS · SESSION 9

Dictionaries

Look values up by name: key-value pairs, methods, nested dictionaries and counting with dictionaries.

Recap

Ask the class:

- Why can a tuple not be changed, but a list can?
- How do you make an **empty set**?
- What does `{1, 2, 3} & {2, 3, 4}` give?

What is a dictionary?

A dictionary stores **key-value pairs** inside curly brackets `{ }`. You find a value by its **key**, not by a position, like looking up a word in a real dictionary.

dictionaries.py

```
student = {"name": "Aman", "age": 20, "city": "Delhi"}
print(student)
print(len(student))
print(type(student))
```

OUTPUT

```
{'name': 'Aman', 'age': 20, 'city': 'Delhi'}
3
<class 'dict'>
```

- **Pairs:** each item is `key: value`, separated by commas.
- **Keys are unique:** a repeated key keeps only the **last** value.
- **Keys must be immutable:** strings, numbers and tuples work, lists do not.
- **Values can be anything:** numbers, text, lists, even other dictionaries.
- **Ordered:** a dictionary remembers the order the items were added.
- **Mutable:** you can add, change and remove pairs.

Ask

In a list you find an item by its **position**. How do you find it in a dictionary?

Making a dictionary

```
empty = {}
a = {"x": 1, "y": 2}
b = dict(x=1, y=2)
c = dict([("x", 1), ("y", 2)])
print(empty, a, b, c)
print(a == b == c)
```

OUTPUT

```
{} {'x': 1, 'y': 2} {'x': 1, 'y': 2} {'x': 1, 'y': 2}
True
```

- `{}` is an **empty dictionary** (that is why an empty set needs `set()`).
- `dict(x=1, y=2)` works when the keys are simple names.
- `dict()` also accepts a list of **(key, value) tuples**.

Repeated keys

Work out the output first, then press Run to check

```
d = {"a": 1, "b": 2, "a": 3}
print(d)
print(len(d))
```

What can be a key

Try it: a list cannot be a key

```
d = {[1, 2]: "list"}
```

ERROR

```
  File "your code", line 1, in <module>
TypeError: unhashable type: 'list'
```

Try it: a tuple can

```
d = {(1, 2): "point", 5: "number", "k": "text"}
print(d[(1, 2)])
```

OUTPUT

```
point
```

Reading a value

Put the **key** in square brackets: `d[key]`.

```
student = {"name": "Aman", "age": 20, "city": "Delhi"}
print(student["name"])
print(student["age"] + 1)
```

OUTPUT

```
Aman
21
```

A missing key

Try it: read the error

```
student = {"name": "Aman", "age": 20}
print(student["city"])
```

ERROR

```
  File "your code", line 2, in <module>
KeyError: 'city'
```

get() is safer

`d.get(key)` gives **None** instead of an error when the key is missing, and you can give a **default** value.

```
student = {"name": "Aman", "age": 20}
print(student.get("name"))
print(student.get("city"))
print(student.get("city", "Unknown"))
```

OUTPUT

```
Aman
None
Unknown
```

Checking for a key

```
student = {"name": "Aman", "age": 20}
print("name" in student)
print("city" in student)
print("Aman" in student)
```

OUTPUT

```
True
False
False
```

Careful

`in` checks the **keys**, not the values. `"Aman" in student` is `False`.

Adding and changing

The **same line** adds a new pair or changes an old one: `d[key] = value`.

```
student = {"name": "Aman", "age": 20}
student["city"] = "Delhi"
student["age"] = 21
print(student)
```

OUTPUT

```
{'name': 'Aman', 'age': 21, 'city': 'Delhi'}
```

update() adds many at once

```
student = {"name": "Aman"}
student.update({"age": 20, "city": "Delhi"})
student.update(age=21)
print(student)
```

OUTPUT

```
{'name': 'Aman', 'age': 21, 'city': 'Delhi'}
```

setdefault()

`setdefault(key, value)` adds the pair **only if the key is missing**, and returns the value that is stored.

```
d = {"a": 1}
print(d.setdefault("a", 99))
print(d.setdefault("b", 2))
print(d)
```

OUTPUT

```
1
2
{'a': 1, 'b': 2}
```

Removing pairs

WAY	WHAT IT DOES
`pop(key)`	removes the key and **returns its value**
`pop(key, default)`	same, but **no error** if the key is missing
`del d[key]`	removes the key
`popitem()`	removes and returns the **last** pair
`clear()`	removes **everything**

```
d = {"a": 1, "b": 2, "c": 3, "d": 4}
print(d.pop("a"))
print(d.pop("z", "not found"))
del d["b"]
print(d)
print(d.popitem())
print(d)
d.clear()
print(d)
```

OUTPUT

```
1
not found
{'c': 3, 'd': 4}
('d', 4)
{'c': 3}
{}
```

Try it: pop with a missing key

```
d = {"a": 1}
d.pop("z")
```

ERROR

```
  File "your code", line 2, in <module>
KeyError: 'z'
```

keys(), values() and items()

Three methods give you the **parts** of a dictionary.

METHOD	GIVES
`d.keys()`	all the **keys**
`d.values()`	all the **values**
`d.items()`	all the **(key, value) pairs**

```
marks = {"maths": 90, "science": 85, "english": 78}
print(list(marks.keys()))
print(list(marks.values()))
print(list(marks.items()))
```

OUTPUT

```
['maths', 'science', 'english']
[90, 85, 78]
[('maths', 90), ('science', 85), ('english', 78)]
```

Handy functions on the values

```
marks = {"maths": 90, "science": 85, "english": 78}
print(sum(marks.values()))
print(max(marks.values()))
print(sum(marks.values()) / len(marks))
print(sorted(marks))
```

OUTPUT

```
253
90
84.33333333333333
['english', 'maths', 'science']
```

Ask

What does `sorted(marks)` sort, the keys or the values?

Looping over a dictionary

A plain `for` loop goes through the **keys**. Use `items()` to get **keys and values** together, with **unpacking** from last class.

```python
marks = {"maths": 90, "science": 85, "english": 78}

for subject in marks:
    print(subject)

for value in marks.values():
    print(value)

for subject, mark in marks.items():
    print(subject, "->", mark)
```

OUTPUT

```
maths
science
english
90
85
78
maths -> 90
science -> 85
english -> 78
```

Finding the best

```python
marks = {"maths": 90, "science": 85, "english": 78}
best = None
for subject, mark in marks.items():
    if best is None or mark > marks[best]:
        best = subject
print(best, marks[best])
print(max(marks, key=marks.get))
```

OUTPUT

```
maths 90
maths
```

Careful

Do not **add or remove keys** while looping over a dictionary. Loop over a copy: `list(d)`.

Copying and joining

Like lists, `b = a` does **not** copy a dictionary. Both names point to the **same** dictionary.

```
a = {"x": 1}
b = a
b["x"] = 99
print(a)

c = a.copy()
c["x"] = 5
print(a, c)
```

OUTPUT

```
{'x': 99}
{'x': 99} {'x': 5}
```

Joining two dictionaries

```
a = {"x": 1, "y": 2}
b = {"y": 20, "z": 30}
print(a | b)
print({**a, **b})
```

OUTPUT

```
{'x': 1, 'y': 20, 'z': 30}
{'x': 1, 'y': 20, 'z': 30}
```

When the same key is in both, the **second** dictionary wins.

Nested dictionaries

A value can be **another dictionary**. Use **one pair of brackets for each level**.

```
students = {
    "Aman": {"age": 20, "city": "Delhi"},
    "Riya": {"age": 19, "city": "Mumbai"},
}
print(students["Riya"])
print(students["Riya"]["city"])
```

OUTPUT

```
{'age': 19, 'city': 'Mumbai'}
Mumbai
```

Changing a nested value

```
students = {
    "Aman": {"age": 20, "city": "Delhi"},
    "Riya": {"age": 19, "city": "Mumbai"},
}
students["Aman"]["age"] = 21
students["Kabir"] = {"age": 22, "city": "Pune"}
print(students["Aman"])
print(len(students))
```

OUTPUT

```
{'age': 21, 'city': 'Delhi'}
3
```

Looping through nested data

```
students = {
    "Aman": {"age": 20, "city": "Delhi"},
    "Riya": {"age": 19, "city": "Mumbai"},
}
for name, info in students.items():
    print(name, "lives in", info["city"])
```

OUTPUT

```
Aman lives in Delhi
Riya lives in Mumbai
```

Ask

How would you read Riya's age from `students`?

Lists and dictionaries together

Real data is often a **list of dictionaries**: one dictionary for each row.

```
people = [
    {"name": "Aman", "marks": 90},
    {"name": "Riya", "marks": 85},
    {"name": "Kabir", "marks": 70},
]
for p in people:
    print(p["name"], p["marks"])

total = 0
for p in people:
    total += p["marks"]
print("Average:", total / len(people))
```

OUTPUT

```
Aman 90
Riya 85
Kabir 70
Average: 81.66666666666667
```

A dictionary with lists as values

```
scores = {"Aman": [90, 80], "Riya": [85, 95]}
scores["Aman"].append(70)
print(scores["Aman"])
for name, marks in scores.items():
    print(name, sum(marks) / len(marks))
```

OUTPUT

```
[90, 80, 70]
Aman 80.0
Riya 90.0
```

Counting with a dictionary

A very common job: **count how many times** each item appears. The item is the **key** and the count is the **value**.

Count the letters

```
word = "banana"
count = {}
for ch in word:
    if ch in count:
        count[ch] += 1
    else:
        count[ch] = 1
print(count)
```

OUTPUT

```
{'b': 1, 'a': 3, 'n': 2}
```

The shorter way with get()

`count.get(ch, 0)` gives `0` for a new letter, so one line does the whole job.

```
word = "banana"
count = {}
for ch in word:
    count[ch] = count.get(ch, 0) + 1
print(count)
```

OUTPUT

```
{'b': 1, 'a': 3, 'n': 2}
```

Think

Walk through the loop for `"banana"` on paper. What is `count` after each letter?

Counting words

Use `split()` to get the words, then count them the same way.

```
text = "the cat and the dog and the bird"
words = text.split()
count = {}
for w in words:
    count[w] = count.get(w, 0) + 1
print(count)
print(len(count))
```

OUTPUT

```
{'the': 3, 'cat': 1, 'and': 2, 'dog': 1, 'bird': 1}
5
```

The most common word

```
text = "the cat and the dog and the bird"
count = {}
for w in text.split():
    count[w] = count.get(w, 0) + 1

top = max(count, key=count.get)
print(top, count[top])
```

OUTPUT

```
the 3
```

Showing the result neatly

```
text = "the cat and the dog and the bird"
count = {}
for w in text.split():
    count[w] = count.get(w, 0) + 1

for w in sorted(count):
    print(w, "-", count[w])
```

OUTPUT

```
and - 2
bird - 1
cat - 1
dog - 1
the - 3
```

Grouping with a dictionary

Instead of a number, the value can be a **list**, so the dictionary **groups** items together.

Group the words by their first letter

```
words = ["apple", "avocado", "banana", "blueberry", "cherry"]
groups = {}
for w in words:
    groups.setdefault(w[0], []).append(w)
print(groups)
```

OUTPUT

```
{'a': ['apple', 'avocado'], 'b': ['banana', 'blueberry'], 'c': ['cherry']}
```

Even and odd

```
groups = {"even": [], "odd": []}
for n in range(1, 8):
    if n % 2 == 0:
        groups["even"].append(n)
    else:
        groups["odd"].append(n)
print(groups)
```

OUTPUT

```
{'even': [2, 4, 6], 'odd': [1, 3, 5, 7]}
```

A quick look at dictionary comprehensions

Just like list comprehensions, you can build a dictionary in **one line**: `{key: value for ... in ...}`.

```
squares = {n: n * n for n in range(1, 6)}
print(squares)

names = ["Aman", "Riya"]
lengths = {name: len(name) for name in names}
print(lengths)
```

OUTPUT

```
{1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
{'Aman': 4, 'Riya': 4}
```

Swapping keys and values

```
d = {"a": 1, "b": 2}
flipped = {v: k for k, v in d.items()}
print(flipped)
```

OUTPUT

```
{1: 'a', 2: 'b'}
```

Sorting a dictionary

```
marks = {"maths": 90, "science": 85, "english": 78}
print(sorted(marks.items()))
print(sorted(marks.items(), key=lambda pair: pair[1]))
```

OUTPUT

```
[('english', 78), ('maths', 90), ('science', 85)]
[('english', 78), ('science', 85), ('maths', 90)]
```

List, tuple, set or dictionary?

	LIST	TUPLE	SET	DICTIONARY
Brackets	`[ ]`	`( )`	`{ }`	`{k: v}`
Find by	index	index	membership	**key**
Duplicates	allowed	allowed	**removed**	keys **unique**
Can change	**Yes**	**No**	**Yes**	**Yes**
Empty one	`[]`	`()`	`set()`	`{}`

Ask

Which one would you use for: a phone book, the days of the week, the unique words in a story, a shopping cart?

Common mistakes

- Using `d[key]` for a key that may be missing: use **`d.get(key)`** instead.
- Thinking `in` checks the **values**: it checks the **keys**.
- Using a **list as a key**: it is not allowed, use a tuple.
- Writing `{}` and expecting a **set**: it is an empty **dictionary**.
- Thinking `b = a` makes a copy: use **`a.copy()`**.
- Changing the dictionary **while looping** over it.

Try it: which line is the mistake?

```
prices = {"apple": 40, "mango": 80}
print(prices["apple"])
print(prices["banana"])
```

```
ERROR
40
  File "your code", line 3, in <module>
KeyError: 'banana'
```

Practice

Warm-up questions

1. Make a dictionary of a book with `title`, `author` and `price`, then print the author.
2. Add a new key `year` to the dictionary, then change the `price`.

3. Print every key and value of a dictionary on its own line.
4. Print the sum of all the values of a dictionary of marks.
5. Check whether the key `"email"` is in a dictionary, without getting an error.

Harder questions

6. Count how many times each **letter** appears in a word.
7. Count how many times each **word** appears in a sentence, and print the most common one.
8. Make a dictionary of the squares of the numbers 1 to 5.
9. Swap the keys and values of a dictionary.
10. Merge two dictionaries and add the values of keys that appear in both.

Find the output

Work out the output first, then press Run to check

```
d = {"a": 1, "b": 2}
d["c"] = 3
d["a"] = 10
print(d.get("z", 0))
print(list(d.values()))
print("b" in d, 2 in d)
d.pop("b")
print(len(d))
```

Try the count question

Your turn: count the vowels in the word, then press Run

```
word = "education"
# make a dictionary that counts each vowel
```

Homework and wrap-up

- Write a program that stores the **names and marks** of 5 students in a dictionary, and prints the **topper** and the **average**.
- Write a program that takes a sentence from the user and prints the **count of each word**.
- Write a program for a small **phone book**: add a contact, search for a contact, and print all contacts.
- Write a program that finds the **first letter that appears only once** in a word, using a dictionary.
- Revise the **list, tuple, set and dictionary** comparison table.

Next class

Functions.